



Trabajo Práctico Especial

72.80 - Big Data

Grupo 8

Luca Maximiliano Rossi, 63730

Patrick Torlaschi, 62273

Nicolás Casella, 62311

Santiago Nartallo, 62208

Federico Viera, 62022

ÍNDICE

1. INTERPRETACIÓN DEL PROBLEMA.....	3
1.1. El problema.....	3
1.2. Usuarios.....	3
1.3. Preguntas principales.....	3
1.4. Objetivos medibles.....	4
2. JUSTIFICACIÓN DE BIG DATA.....	4
2.1. Volumen.....	4
2.2. Velocidad.....	4
2.3. Variedad.....	4
2.4. Veracidad.....	5
2.5. Valor.....	5
3. INVENTARIO Y PERFIL DE LAS FUENTES.....	5
3.1. Inventario.....	5
3.2. Principales hallazgos de calidad y riesgos.....	6
3.3. Orden de llegada de los eventos.....	7
3.4. Trazabilidad.....	7
4. ARQUITECTURA DE ALTO NIVEL.....	8
4.1. Diagrama de arquitectura.....	8
4.2. Patrón arquitectónico.....	9
4.3. Capacidades transversales.....	10
5. MATRIZ REQUISITO-COMPONENTE.....	10
6. DISEÑO DEL DATA LAKE.....	11
6.1. Zonas.....	11
6.2. Formatos y particionamiento.....	12
6.3. Metadatos.....	13
6.4. Reglas de calidad y cuarentena.....	13
6.5. Promoción entre zonas.....	14
6.6. Retención.....	15
6.7. Naming.....	16
7. FLUJOS DE DATOS BATCH Y STREAMING.....	17
7.1. Flujo batch.....	17
7.2. Flujo streaming.....	18
7.3. Watermark.....	18
7.4. Serving.....	18
8. FLUJO BATCH CON LÓGICA MAPREDUCE.....	19
9. SUPUESTOS, RIESGOS Y DECISIONES ABIERTAS.....	19
9.1. Supuestos.....	19
9.2. Riesgos y mitigaciones.....	20
9.3. Decisiones abiertas.....	20
10. ESFUERZO, ROLES Y RECURSOS.....	21

1. INTERPRETACIÓN DEL PROBLEMA

1.1. El problema

El cloud provider recibe datos de uso, costo, facturación y soporte en dos modalidades: eventos continuos de consumo (*near* real-time) y Master Data diaria o mensual. Los datos llegan crudos y con nulos, tipos ambiguos, costos negativos, y no existe una fuente confiable desde la cual FinOps, Soporte, y Producto puedan responder sus preguntas de negocio.

El objetivo es construir un pipeline reproducible e idempotente que lleve los datos desde Landing hasta marts analíticos en Cassandra/AstraDB, a través de un Data Lake en Parquet con zonas Bronze, Silver y Gold, y que resuelva la evolución de esquema, los tipos ambiguos, los nulos y las anomalías de costo. Se resolverá cuando las consultas de negocio definidas puedan responderse desde Cassandra con evidencia y la solución pueda ejecutarse desde un entorno limpio.

1.2. Usuarios

Usuario	Necesidad analítica	Granularidad	Frecuencia
FinOps	Costos, consumo, revenue, créditos, impuestos, anomalías y eficiencia por organización y servicio	org × fecha × servicio; org × mes (revenue)	Near real-time (consumo), mensual (facturación)
Soporte	Volumen de tickets, severidad, cumplimiento de SLA y CSAT por organización y fecha	org × fecha × severidad	Batch diario
Producto / Usage	Uso de servicios, requests, métricas operativas, tokens GenAI y carbono cuando estén disponibles	org × fecha × servicio; org × fecha (tokens)	Near real-time

Tabla 1. Usuarios y sus necesidades

1.3. Preguntas principales

1. Costos y requests diarios por organización y servicio en un rango de fechas (FinOps × *org_daily_usage_by_service*).
2. Top-N servicios por costo acumulado en los últimos 14 días para una organización (FinOps × *org_daily_usage_by_service*).
3. Evolución de tickets críticos y tasa de SLA breach por día en los últimos 30 días (Soporte × *tickets_by_org_date*).

4. Revenue mensual con créditos e impuestos, normalizado a USD (FinOps $\times$ *revenue_by_org_month*).
5. Tokens GenAI y costo estimado por día, cuando existan (Producto $\times$ *genai_tokens_by_org_date*).

1.4. Objetivos medibles

- **Ingesta trazable:** todos los registros de Landing presentes en Bronze o en quarantine, con *ingest_ts* y *source_file*.
- **Conciliación:** Bronze = Silver + quarantine en cada ejecución.
- **Calidad explícita:** reglas activas con conteos y muestras.
- **Idempotencia:** conteos antes y después de re-ejecutar, sin duplicados.
- **Frescura:** tiempo de Landing a Bronze para un archivo nuevo.
- **Respuesta al negocio:** las 5 consultas principales se responden desde Cassandra con CQL y resultados.

2. JUSTIFICACIÓN DE BIG DATA

2.1. Volumen

Cada recurso de cada cliente emite eventos de uso de manera continua, por lo que es esperable que el volumen de datos crezca rápidamente a medida que se sumen clientes y recursos. Este volumen exige procesamiento distribuido (Spark), almacenamiento columnar particionado (Parquet) y una base distribuida para el serving (Cassandra).

2.2. Velocidad

El enunciado exige métricas de uso, consumo y costos incrementales en near real-time. Por eso los eventos se procesan con Structured Streaming en micro-lotes, y Silver y Gold se recalculan para las fechas afectadas después de cada ingesta. La Master Data, por otro lado, aparece en forma de fotos diarias o mensuales y permite procesamiento batch.

2.3. Variedad

Contamos con siete CSV como fuentes estructuradas y un flujo JSONL semi-estructurado con dos versiones de esquema: la v2 agrega *carbon_kg* y, para *genai*, *genai_tokens*. El campo *value* llega como número, como texto o nulo. Hay además JSON embebido (*tags_json*) y tres monedas.

2.4. Veracidad

Los datos llegan con nulos, tipos ambiguos, entre otros problemas de calidad (ver 3.2 *Hallazgos de calidad*). Para que los datos del negocio sean confiables, se aplican reglas de calidad: los registros marcados como inválidos van a quarantine, los recuperables se corrigen o imputan y los sospechosos se conservan marcados con un flag.

2.5. Valor

Los datos responden las necesidades de cada dominio: costo por organización, servicio y día, y detección de anomalías de costo para FinOps; evolución de tickets críticos y cumplimiento de SLA para Soporte; adopción de GenAI y huella de carbono para Producto.

3. INVENTARIO Y PERFIL DE LAS FUENTES

Llamamos "Master Data" a los siete CSV que llegan por batch y eventos de uso al flujo JSONL. Las fuentes se perfilaron con el script *notebooks/00_exploracion_datos.ipynb* que está subido al repositorio.

3.1. Inventario

- **customers_orgs.csv**: 80 filas; una fila por organización cliente; clave org_id; batch diario. Es la dimensión maestra de organizaciones: todas las otras fuentes y marts se vinculan a ella por org_id.
- **users.csv**: 800 filas; una fila por usuario; clave user_id; batch diario. Alimenta Silver.
- **resources.csv**: 400 filas; una fila por recurso cloud; clave resource_id; batch diario. Se cruza con los eventos.
- **support_tickets.csv**: 1000 filas; una fila por ticket; clave ticket_id; batch diario. Alimenta tickets_by_org_date.
- **marketing_touches.csv**: 1500 filas; una fila por interacción de marketing; clave touch_id; batch diario. Llega hasta Silver.
- **nps_surveys.csv**: 92 filas; una fila por encuesta; clave (org_id, survey_date); batch diario. Alimenta el NPS histórico en Silver.
- **billing_monthly.csv**: 240 filas; una fila por factura mensual; clave invoice_id; batch mensual. Alimenta revenue_by_org_month.

- **usage_events_stream/*.jsonl:** 43200 eventos en 120 archivos; un evento de consumo por línea; clave event_id; streaming. Alimenta los marts de uso, anomalías y GenAI.

3.2. Principales hallazgos de calidad y riesgos

Fuente	Hallazgo	Casos	Riesgo
usage_events	cost_usd_increment < -0.01	211 (0.5%)	Resta costo real, distorsiona totales, top-N
usage_events	Picos de costo > 3 * p99 del servicio	79 (0.2%)	Sesgan promedios y umbrales de anomalía
usage_events	unit nulo con value presente	2038 (4.7%)	Métricas sin unidad
usage_events	Value como texto/nulo	1309 (3%) y 877 (2%)	Sumas de uso incorrectas o con huecos por falla de cast
usage_events	Eventos v1 sin carbon_kg ni genai_tokens	10800 (25%)	Tratar nulos como error o leer nulos como 0
billing_monthly	USD con tipo de cambio != 1	160 (66.7%)	Todas las facturas en USD
billing_monthly	Organizaciones con más de una moneda	50 (62.5%)	No se puede asumir una moneda por organización
billing_monthly	credits nulo	137 (57.1%)	Si nulo no significa “sin crédito”, el revenue queda sobreestimado
billing_monthly	Subtotal negativo	13 (5.4%)	Revenue mensual reducido o inconsistente
support_tickets	Csat fuera de 1-5	40 (4%)	CSAT promedio sesgado
customer_orgs	Nps_score nulo o fuera de [-100,100]	11 (13.8%) y 1 (1.3%)	Atributos incompletos en NPS histórico
users	last_login anterior a created_at	232 (29%)	Actividad de usuarios poco confiable

Otras	Plan enterprise con is_enterprise falso; converted sin clicked	9 (11.3%) y 96 (6.4%)	Segmentación inconsistente
Todas	Claves duplicadas, huérfanos, event_id nulo o duplicado, JSON inválido	0	Sin hallazgos

Tabla 2. Hallazgos de calidad y riesgos

3.3. Orden de llegada de los eventos

Cada uno de los 120 archivos cubre el rango de 60 días, de modo que el orden de llegada no sigue el tiempo de evento: el 99 % de los eventos es anterior al máximo ya visto, con un retraso (diferencia entre timestamp máximo en archivos previos y timestamp del evento) mediano de 30 días. Este hallazgo condiciona el watermark.

3.4. Trazabilidad

Las fuentes no incluyen metadatos de origen; cada registro se identifica sólo por su clave natural y Landing se conserva inmutable. Bronze agrega source_file, ingest_ts y run_id. Desde cualquier fila de Gold o de Cassandra se llega por las claves y el run_id a los registros de Silver, y de ahí a los archivos de origen.

4. ARQUITECTURA DE ALTO NIVEL

4.1. Diagrama de arquitectura

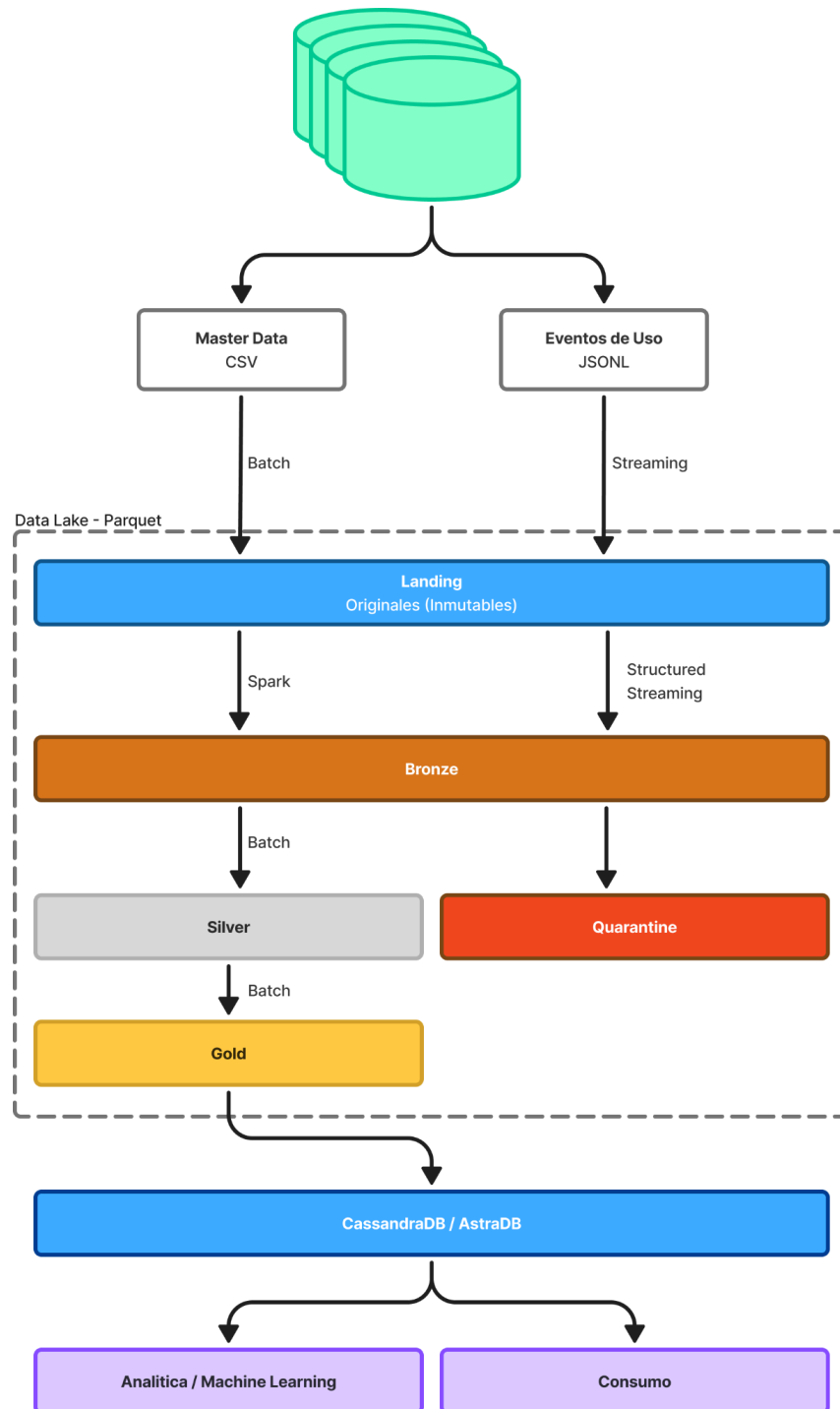


Figura 1. Arquitectura de alto nivel.

Fuentes e ingesta

- **Master Data:** siete CSV que ingresan por batch con Spark.
- **Eventos de uso:** archivos JSONL que ingresan en micro-lotes con Structured Streaming.

Data Lake

- **Landing:** datos originales sin modificar.
- **Bronze:** datos tipados y trazables (run_id, ingest_ts, source_file).
- **Silver:** reglas de calidad, joins con la Master Data, cálculo de features y detección de anomalías. Los registros inválidos van a quarantine.
- **Gold:** marts de negocio por dominio.

Serving y consumo

- **Cassandra/AstraDB:** publica los marts en el keyspace cloud_analytics, con tablas diseñadas por consulta.
- **Analítica/ML:** análisis de los marts de negocio, costos y anomalías detectadas en Silver.
- **Consumo:** dashboards y consultas CQL para FinOps, Soporte y Producto.

4.2. Patrón arquitectónico

Se adopta un patrón híbrido: la velocidad se resuelve en la ingesta y toda la lógica de negocio vive en una única capa batch.

- **Ingesta streaming:** los eventos llegan a Bronze en micro-lotes, con watermark, deduplicación por event_id y checkpoint.
- **Ingesta batch:** la Master Data se carga a Bronze en ejecuciones diarias o mensuales.
- **Procesamiento batch común:** Silver y Gold se calculan en batch sobre Bronze para ambas fuentes; un evento tardío sólo obliga a recalcular su partición de fecha.

Justificación de nuestra elección frente a los distintos patrones de arquitectura posibles:

- **Lambda** se descarta por dos motivos:
 - Duplica la lógica de negocio, haciendo que los mismos marts se calculen dos veces, en una capa batch y en una de velocidad, logrando que se tengan que mantener ambas implementaciones de forma consistente.
 - La capa de velocidad no aporta para este problema porque, dado que su objetivo es mostrar resultados provisionales de forma rápida, para este caso, cada archivo trae eventos de 60 días y casi todos llegan desordenados, por lo que esos resultados cambiarían con cada archivo nuevo.

- **Kappa** se descarta porque obligaría a tratar como streams los siete CSV provistos, que contienen fotos diarias o mensuales y no requieren de una baja latencia. Además, agregar los eventos dentro del stream exigiría un watermark que cubra el atraso máximo observado (de aproximadamente 60 días), por lo que ningún día podría cerrarse antes de ese plazo y el estado debería mantener abiertas las sumas de todos los días de la ventana para cada organización y servicio. Recalcular por partición en batch evita ese estado.
- El patrón híbrido nos permite tomar lo necesario de cada enfoque. Del streaming tomamos la ingesta, la deduplicación y el checkpointing. Del batch tomamos un único cálculo de Silver y Gold, recalculado por partición. Además, ante un cambio de reglas o carga de evento muy tardío, el pipeline se re-ejecuta desde Landing, que es inmutable y se conserva sin límite de tiempo.

4.3. Capacidades transversales

- **Gobierno:** responsable por zona y dataset, Landing de solo lectura y diccionario de datos versionado.
- **Calidad:** reglas de calidad con conteos por regla y quarantine con motivo y muestras.
- **Metadatos y linaje:** ingest_ts, source_file y run_id en Bronze, dq_flags en Silver y linaje documentado de fuente a mart.
- **Observabilidad:** conteos por etapa, progreso del stream (lastProgress) y log por ejecución.

5. MATRIZ REQUISITO-COMPONENTE

Requerimiento	Solución técnica	V de Big Data
Uso y costos near real-time	Ingesta streaming a Bronze con Structured Streaming	Velocidad
Eventos tardíos y desordenados	Watermark de 61 días y recálculo en batch por fecha	Velocidad y veracidad
Batch de Master Data	Ingesta batch a Bronze con spark.read.csv	Volumen y variedad
Almacenamiento escalable	Data Lake por zonas en Parquet particionado	Volumen

Esquema explícito y versiones v1/v2	Bronze y Silver con un StructType unificado	Variedad
Calidad y aislamiento de inválidos	Reglas de calidad y quarantine en PySpark	Veracidad
Normalización y enriquecimiento	Silver con broadcast join a la Master Data	Variedad
Features de uso y costo	Paso de Silver a Gold con groupBy y agg	Valor
Anomalías de costo	Analítica/ML con percentiles, MAD y z-score	Veracidad y valor
Marts por dominio	Gold en Parquet particionado por fecha	Valor
Consultas de baja latencia	Serving en Cassandra/AstraDB	Velocidad y valor
Reproceso sin duplicados	Checkpoint, dedupe, overwrite y upsert	Veracidad
Trazabilidad y linaje	Columnas técnicas y diccionario de datos	Veracidad
Performance	Particiones por fecha, coalesce y broadcast en Spark	Volumen

Tabla 3. Matriz Requisito Componente

6. DISEÑO DEL DATA LAKE

El DataLake se organiza en zonas con niveles crecientes de procesamiento. Para cada una indicamos su granularidad, es decir que representa una fila en la zona.

6.1. Zonas

Landing

Contiene los archivos originales sin modificar. Tiene un estado inmutable y un tiempo de retención indefinido. Es la fuente de verdad para el procesamiento y reprocesamiento del pipeline, ante la eventualidad de un cambio.

Bronze

Es la primera zona en Parquet. Preserva el nivel de granularidad que había en la fuente, donde cada fila es un registro original. No se realizan agregaciones sobre los datos todavía. Se castean los datos al esquema explícito, se eliminan duplicados por clave natural y se agregan las columnas de trazabilidad (ingest_ts, source_file y run_id). Los eventos se particionan por event_date mientras que la Master Data se guarda como un snapshot en cada ejecución, particionada por ingest_date.

Silver

Mantiene la granularidad de *Bronze* pero con datos limpios y enriquecidos. Se corrigen nulos y tipos, unifican las versiones v1 y v2 del esquema y se cruzan los eventos con la Master Data. Cada corrección queda marcada en `dq_flags` y los registros inválidos se envían directo a quarantine.

Gold

Contiene los marts de negocio. A diferencia de las zonas anteriores, cada fila es un agregado y la granularidad depende de cada consulta que responde cada mart. Ej *org_daily_usage_by_service* donde cada fila resume el uso de una organización en un día para un servicio. Estos marts luego se cargan a Cassandra.

Quarantine

Guarda todos los registros que no superan las reglas de calidad, junto con la regla que falló y su origen para luego poder ser revisados y reprocesados si se aplicara.

Checkpoints

Guarda el estado del streaming que spark administra automáticamente. Permite que si el proceso se interrumpe o se vuelve a ejecutar, no se procesen dos veces los mismos archivos ni se generen duplicados.

6.2. Formatos y particionamiento

Landing conserva el formato original de cada fuente. A partir de Bronze, todos los datos se guardan en Parquet con compresión snappy. Esto permite leer solo las columnas que usa cada consulta y admite columnas opcionales, lo que resuelve la convivencia de las versiones v1 y v2 del esquema. Además, los datos se particionan por fecha, ya que casi todas las consultas se filtran por un periodo. De esta forma, Spark solo lee carpetas de un rango pedido y el reproceso solo reescribe únicamente las fechas afectadas.

Dataset	Zona	Partición	Motivo
Master Data	Bronze	ingest_date	Cada ejecución es una foto completa; Por lo que se puede reconstruir el estado en cualquier fecha de forma idempotente.
billing_monthly	Silver	billing_month	Las consultas de facturación son mensuales
usage_events	Bronze y Silver	event_date	Las consultas se filtran por rango de fechas.

Dataset	Zona	Partición	Motivo
Marts	Gold	usage_date, date o month	Permite consultar por periodo y cargar a Cassandra solo las fechas nuevas.
Registros Inválidos	Quarantine	ingest_date	Permite revisar que fallo en cada ejecución

Tabla 4. Formato y Particionamiento

No se particiona por servicio, dado que 6 servicios en 60 días generaron 360 carpetas de unas 120 filas cada una, y tantos archivos chicos ralentizan la lectura. Por el mismo motivo, en streaming se utiliza coalesce por micro-lote para escribir pocos archivos por fecha.

6.3. Metadatos

Además de los datos de negocio, cada zona agrega columnas que permiten saber de dónde viene cada registro, que correcciones se le aplicaron y en que ejecución se generó. Estas columnas son la base de la trazabilidad del pipeline.

Columna	Zona	Contenido
ingest_ts, source_file	Bronze, Silver y Quarantine	Momento de ingesta y archivo de origen
run_id	Todas desde Bronze	Id de la ejecución que generó el registro
dq_flags	Silver	Correcciones y alertas aplicadas al registro.
is_cost_anomaly, anomaly_score	Silver y Gold	Resultado de la detección de anomalías
valid_from, valid_to, is_current	Silver (orgs_scd2)	Periodo de vigencia de cada versión de una organización.
_rule_id, _quarantine_reason, _raw	Quarantine	Regla incumplida, motivo y registro original

Tabla 5. Metadatos

Las filas de Gold no guardan source_file porque cada una resume eventos de varios archivos. Para las trazas se usa su run_id y sus claves (organización, fecha y servicio) para llegar a los registros de Silver de donde se originaron, y luego al archivo origen.

6.4. Reglas de calidad y cuarentena

Cada regla tiene una acción según la gravedad del problema. Si el registro no se puede usar, va a quarantine. Si el problema se puede corregir de forma confiable, se corrige y se marca en dq_flags. Si el dato es válido pero llamativo, se conserva con un flag.

Regla	Dataset	Si falla
event_id no nulo	usage_events	Quarantine
event_id no duplicado	usage_events	Se descarta el duplicado.
schema_version es 1 o 2 y sus campos son coherentes	usage_events	Quarantine
Value numerico	usage_events	Si es nulo, flag; si es texto numérico, cast y flag; si no es numérico, quarantine
Unit no nulo cuando hay value	usage_events	Se imputa desde metric y se marca; si no es posible, quarantine
cost_usd_increment mayor o igual a 0	usage_events	Quarantine
Clave natural no nula	Master Data	Quarantine
NPS entre -100 y 100	customers_orgs	Se reemplaza por NULL y se marca
CSAT entre 1 y 5	support_tickets	Se reemplaza por NULL y se marca
last_login posterior a created_at	users	Flag
Credits no nulo	billing_monthly	Se reemplaza por 0
Subtotal no negativo y tipo de cambio de USD igual a 1	billing_monthly	Flag de anomalía.

Tabla 6. Reglas de calidad y accionar

Por cada regla se registra la cantidad de registros afectados y se exporta una muestra de 20 filas, lo que permite medir la calidad de cada ejecución y revisar los casos que fallaron.

6.5. Promoción entre zonas

Un dato solo avanza a la zona siguiente si cumple dadas condiciones. Así, cada zona garantiza un nivel de calidad mayor que la anterior y ningún registro se pierde en el camino. O avanza, o queda en cuarentena con el motivo respectivo.

Paso	Condición	Si no se cumple
Landing → Bronze	El registro se puede leer, tiene clave y sus tipos se pueden convertir.	Quarantine estructural
Bronze → Silver	El registro cumple las reglas de calidad o pudo corregirse.	Quarantine con la regla incumplida
Silver → Gold	Los conteos de la ejecución cierran.	No se publica Gold, se registra en el log
Gold → Cassandra	El mart está completo para las fechas procesadas.	No se carga y se reintenta en la próxima ejecución.

Tabla 7. Promociones entre Zonas

Antes de publicar Gold, se controla que no se haya perdido ningún registro en el camino. Todo lo que llegó a Landing tiene que estar en Bronze, en cuarentena o haber sido descartado como duplicado. Y todo lo que está en Bronze tiene que haber pasado a Silver o Quarantine. Si el conteo no se verifica correctamente, la ejecución se frena antes de llegar a Gold.

La carga a Cassandra se hace por fecha y reemplaza los registros existentes con la misma clave, por lo que se puede repetir sin tener duplicados.

6.6. Retención

Cada zona conserva los datos el tiempo necesario para su uso. Como Landing se guarda indefinidamente los datos y desde ahí se puede reprocesar todo el pipeline, las zonas intermedias pueden tener una retención más corta sin presentar pérdidas de información.

Zona	Retención	Motivo
Landing	Indefinida	Es la fuente de verdad para reprocesar ante cambios de reglas o eventos tardíos
Bronze	30 fotos de Master Data 13 meses de eventos	Las fotos permiten re-ejecutar el último mes; la historia de clientes ya queda guardada en orgs_scd2
Silver	13 meses orgs_scd2 completa	Permite comparar cada mes con el mismo mes del año anterior y conservar la historia completa de cada cliente
Gold	25 meses	Permite analizar tendencias de dos años completos más el mes en curso
Checkpoints	Mientras el stream esté activo	Si se borran, los eventos se vuelven a procesar desde Landing
Quarantine	90 días y muestras permanentes	Da tiempo a investigar y corregir los errores, y las muestras quedan para auditoría

Tabla 8. Retención de datos por Zona

6.7. Naming

Todos los nombres siguen convenciones fijas, para que la ubicación y el contenido de cada dataset se puedan deducir a partir de su nombre.

Rutas

Cada dataset se guarda con una ruta con la forma:

`{zona}/{dataset}/{partición}=valor/`

Por ejemplo, *bronze/usage_events/event_date=2025-09-01*. La ruta base es configurable para que el mismo código funcione en Colab o en otro entorno. Landing conserva los nombres originales de los archivos, quarantine replica la estructura de su dataset de origen y los checkpoints se guardan en *checkpoints/{job}/*.

Datasets

En Bronze y Silver se usa el nombre de la fuente sin extensión. Los datasets derivados agregan un sufijo que indica su tipos, como *orgs_scd2*. En Gold, el nombre describe el contenido y la granularidad del mart, por ejemplo *org_daily_usage_by_service*.

Columnas

Todos los nombres van en minúscula y snake_case. Las fechas terminan en *_date*, los timestamps en *_ts* y los montos normalizados en *_usd*. Los flags booleanos empiezan con *is_*. Las columnas técnicas de cuarentena empiezan con guión bajo (*_rule_id*, etc) para distinguirlas fácilmente.

Serving

El keyspace de Cassandra es *cloud_analytics*, y cada tabla tiene el mismo nombre que el mart de Gold del que se carga, lo que permite rastrear saber cuál fue el origen.

7. FLUJOS DE DATOS BATCH Y STREAMING

La Figura 2 muestra los dos flujos de ingesta y su convergencia en Silver; las tablas siguientes detallan cada paso.

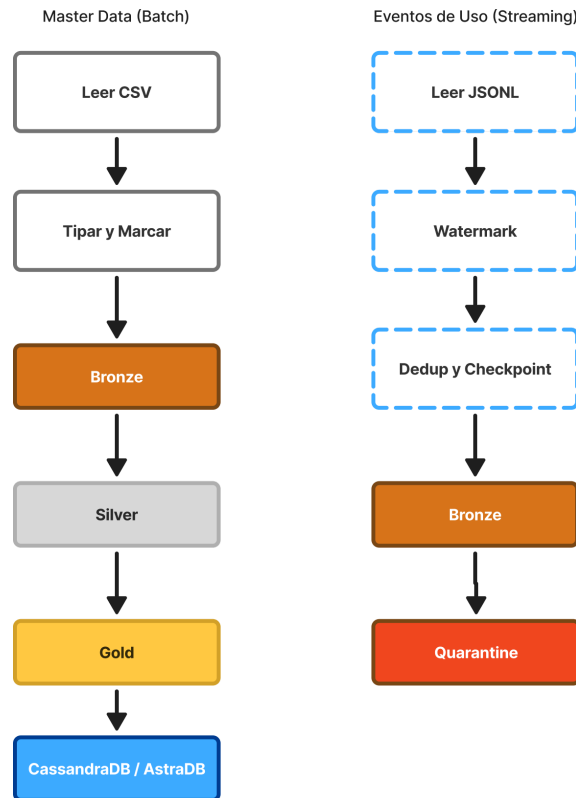


Figura 2. *Flujos batch y streaming desde Landing hasta Cassandra*

7.1. Flujo batch

1. Lectura de los CSV con esquema explícito, en modo PERMISSIVE (spark.read.csv) , resultando en DataFrame tipado.
2. DataFrame trazable: Cast con fallback y agregado de columnas técnicas (withColumn, to_date).
3. Bronze: Dedupe por clave natural; las filas sin clave van a quarantine (dropDuplicates), particionado por ingest_date.
4. Silver: Normalización, validación de rangos, credits nulo a 0, revenue_usd, SCD2.
5. Gold: Agregación por el grano de cada mart (groupBy / agg), con overwrite dinámico.
6. Carga por partición y consultas CQL (conector Spark o foreachBatch con el driver) a Cassandra y AstraDB.

7.2. Flujo streaming

1. Lectura de eventos con `readStream.format("json")`, con trigger.
2. Esquema unificado v1+v2: value como string y carbon_kg y genai_tokens nullable (StructType).
3. Cálculo de event_ts y watermark de 61 días (withWatermark).
4. Deduplicación por event_id con checkpoint (dropDuplicatesWithinWatermark).
5. Cast de value, quarantine estructural, columnas técnicas y coalesce por event_date (foreachBatch) a Bronze y quarantine.
6. Recálculo de Silver y Gold para las fechas afectadas, en batch.

7.3. Watermark

El watermark define cuánto tiempo se espera un evento atrasado antes de descartarlo. Como los archivos llegan desordenados (sección 3.3), un watermark de pocos minutos descartaría casi el 99 % de los eventos a partir del segundo micro-lote.

Por eso se fija en 61 días, que es el período completo del dataset más un día de margen. Con este valor no se pierde ningún evento. Además, el streaming no calcula agregados: solo deduplica y escribe en Bronze, y los totales se recalculan después en batch para las fechas afectadas.

En producción, el margen se ajustaría según los retrasos reales de los eventos (por ejemplo, 48 horas), y los eventos que lleguen más tarde se reprocesarían desde Landing.

7.4. Serving

El keyspace cloud_analytics tiene una tabla por consulta (query-first), con org_id como clave de partición y la fecha como clustering.

Consulta	Tabla	Clave primaria
1 y 2	org_daily_usage_by_service	((org_id), usage_date, service)
2	top_services_by_org_14d	((org_id, as_of_date), rank)
3	tickets_by_org_date	((org_id), date, severity)
4	revenue_by_org_month	((org_id), month)
5	genai_tokens_by_org_date	((org_id), date)
	cost_anomaly_mart	((org_id), date, service)

Tabla 9. Presentación de datos

8. FLUJO BATCH CON LÓGICA MAPREDUCE

El flujo batch de referencia es el cálculo de *org_daily_usage_by_service*, mart central de FinOps.

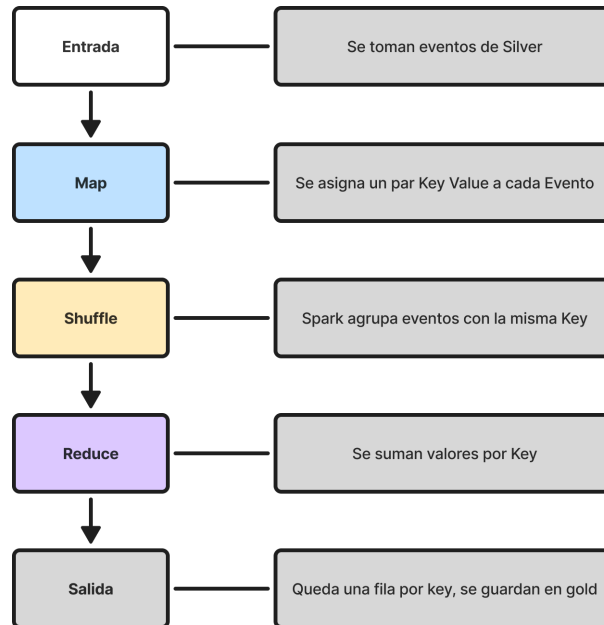


Figura 3. Cálculo de *org_daily_usage_by_service* en fases MapReduce

Fases

- **Map:** cada evento válido emite la clave (*org_id*, *usage_date*, *service*) y un vector con costo, requests, *cpu_hours*, *storage_gb_hours*, tokens y carbono.
- **Shuffle:** los pares con la misma clave se agrupan en el mismo reducer.
- **Reduce:** se suman los vectores componente a componente y se obtiene una fila por organización, día y servicio.

9. SUPUESTOS, RIESGOS Y DECISIONES ABIERTAS

9.1. Supuestos

- Las ventanas se anclan en el 31/08/2025. Si no, las queries 2 y 3 no devuelven nada.
- Los timestamps y *usage_date* están en UTC.
- *cost_usd_increment* está expresado en USD. Si no, requiere conversión con FX.

- Asumimos que el revenue usd se calcula de la siguiente forma:
 $revenue_usd = (subtotal - credits + taxes) \times exchange_rate_to_usd$.
De manera tal que quede correctamente normalizado.
- Un ticket crítico si y sólo si, tiene severity “critical” en el csv de support tickets, no se consideran críticos a los tickets de severity “high”.
- El NPS (Net Promoter Score) está en escala -100 a 100, dado a que en el dataset se encontraban entradas negativas y estas serían inválidas.
- El costo GenAI es la suma de *cost_usd_increment* de los eventos genai.
- Los créditos se informan solo en *billing_monthly*.

9.2. Riesgos y mitigaciones

- **Un watermark corto descarta aproximadamente 99 % de los eventos** (probabilidad alta, impacto alto): margen de 61 días y agregación en batch.
- **Cambios en las reglas o en el esquema:** Agregado de columnas extras opcionales.
- **Latencia sin broker dedicado** (alta, medio): micro-lotes chicos; Kafka en producción.
- **Quarantine excesiva por unit nulo** (alta, medio): imputación determinística.

9.3. Decisiones abiertas

- **Tratamiento del tipo de cambio:** en varias facturas en USD el tipo de cambio es distinto de 1. Se propone usar el valor provisto y marcar esos casos con un flag de anomalía, en lugar de corregirlos.
- **Componente de Analítica/ML:** se propone la detección de anomalías de costo, ya que responde a una necesidad directa de FinOps. Se ejecuta sobre Gold y su resultado se publica como *cost_anomaly_mart*.
- **Umbral de anomalía:** se propone marcar como anómalo un costo diario mayor a 3 veces el percentil 99 de su servicio. El valor final se ajustará al aplicarlo sobre el mart, revisando cuántos casos detecta y si son razonables.
- **Métricas como columnas o map<text, double>:** las métricas de uso pueden guardarse como columnas fijas (*cpu_hours*, *requests*, etc.) o como un único campo map con clave y valor. Se proponen columnas fijas, porque son más simples de consultar en Cassandra y la cantidad de métricas es conocida. Un map solo convendría si aparecieran métricas nuevas con frecuencia.

10. ESFUERZO, ROLES Y RECURSOS

Rol	Responsabilidades
Arquitectura y coordinación	Diagrama, decisiones, integración, README y documentación
Ingesta batch	Landing → Bronze de la Master Data, esquemas y columnas técnicas
Ingesta streaming	Landing → Bronze de eventos, watermark, dedupe y checkpoint
Calidad, Silver y gobierno	Reglas, quarantine, joins, SCD2, diccionario y linaje
Gold, serving y analítica	Marts, Cassandra, consultas CQL y anomalías

Tabla 10. División de tareas y responsabilidades

La estimación se hizo por bloque de trabajo, según la complejidad que requiera cada uno y se agrega un margen de horas adicionales para posibles riesgos o dificultades con las que nos podamos llegar a encontrar.

Bloque	Horas-persona
Correcciones de la 1ª entrega y configuración del entorno	12
Ingesta batch a Bronze	14
Ingesta streaming a Bronze	18
Silver, reglas de calidad y quarantine	24
Gold, carga a Cassandra y dos consultas	18
Idempotencia, gobierno, pruebas, README y backlog	16
Margen de riesgo (15% del esfuerzo para la entrega correspondiente)	15
Subtotal hasta la 2ª entrega	117
Marts restantes y cinco consultas	20
Analítica/ML	14
Documentación, presentación, video y defensa	24
Margen de riesgo (15% del esfuerzo para la entrega correspondiente)	9
Total	184

Tabla 11. Esfuerzo en horas persona

El total equivale a unas 37 horas por integrante. Los recursos (Google Colab, Google Drive, Python (PySpark), AstraDB en su plan gratuito y GitHub) no tienen costo para el volumen de datos disponibles hasta el momento.